

ContextBridge

Memory Methodology - Team Guide v1

A simple explanation of how ContextBridge decides what to remember, how memory changes, and how an AI agent gets the right memory later.

Document	Value
Audience	Team members
Purpose	Explain the chosen methodology, not just list components
Language	Simple and implementation-oriented
Version	v1 methodology baseline

How to read this guide

The easiest way to understand ContextBridge is to follow one piece of information through its whole life: conversation -> candidate memory -> decision -> stored memory -> later retrieval. Each section explains what we chose, why we chose it, and where the idea came from.

Important: this is a synthesis, not a copy

No single paper gives us the complete design. We combined ideas that solve different failure modes. Where a choice comes from a paper, the paper is named. Where it is an engineering decision made for our project, it is marked as engineering rationale.

1. First, what problem are we solving?

Imagine this sequence:

1. Today you work with Agent A in VS Code and explain that your project uses PostgreSQL and that you prefer TypeScript.
2. Tomorrow you open another IDE or start a fresh session with Agent B.
3. The new agent has no reliable access to the earlier context.
4. You have to explain the same project decisions and preferences again.

ContextBridge adds an external memory layer that belongs to the developer/project rather than to one model session.

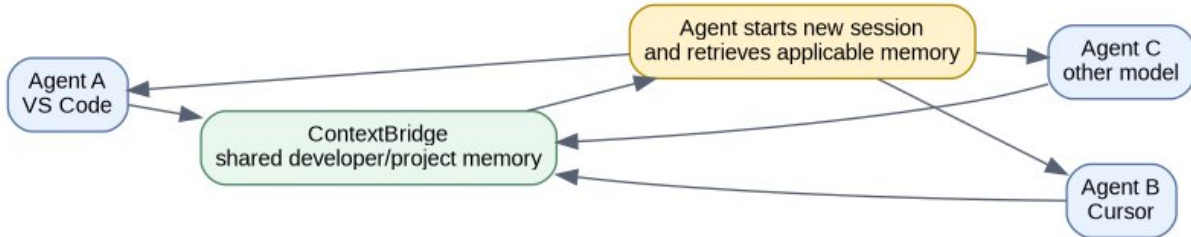


Figure 1. Different agents can use the same persistent developer/project memory.

The important idea is not "chat history forever." The goal is durable, useful knowledge that can be shared across agents and sessions.

Source basis: the survey defines agent memory as persistent and evolving; Mem0, Zep and related systems show external memory can sit between interaction and the model. [P1][P2][P3]

2. The whole methodology in one picture

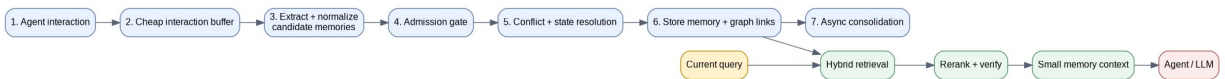


Figure 2. ContextBridge end-to-end memory lifecycle.

Read the flow from left to right. The left side is mainly about writing memory. The right side is mainly about using memory.

Stage	Simple question	Why it exists
Capture	What happened in the session?	Keep a faithful source buffer without expensive processing every turn.
Extract + normalize	What could become durable knowledge?	Turn conversation into clean, self-contained candidates.
Admission	Is this worth remembering?	Avoid storing noise, unsupported claims and duplicates.
Conflict + state	Does this change or contradict something we already know?	Keep the correct current state without losing history.
Store + graph	How are facts connected?	Support provenance, relationships and later expansion.
Consolidate	Can related memories be organized more efficiently?	Spend expensive processing only when useful.
Retrieve	What memory is relevant to this query?	Use semantic, exact and structured signals together.
Verify + assemble	Can we trust this retrieved set, and how much should we show?	Reduce wrong-memory use and long-context noise.

3. Scope: developer memory vs project memory

We separate two kinds of durable information:

Scope	Example	Default behavior
Developer	"I prefer TypeScript for frontend work."	Can apply across projects.
Project	"RozgaarSetu uses Clerk authentication."	Used first for questions about RozgaarSetu.

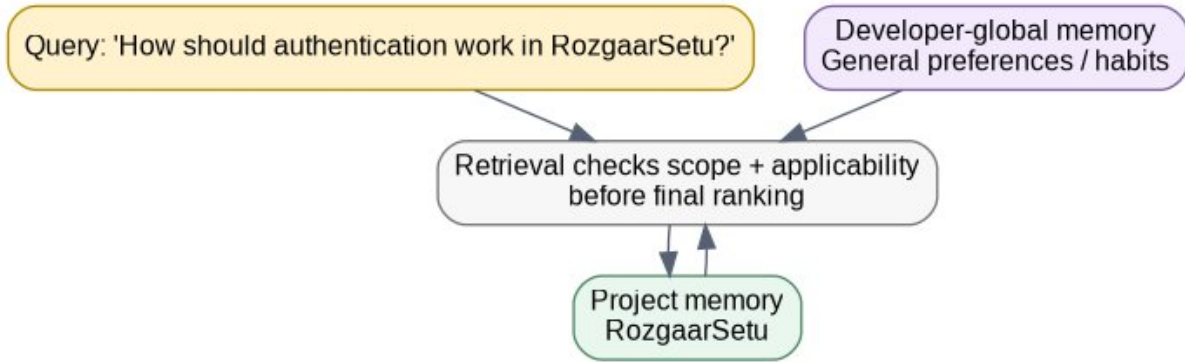


Figure 3. Scope is a retrieval signal, not just a label.

Why this decision?

- Cross-agent memory should follow the developer, but project facts should not leak into unrelated projects. This is why scope is part of every memory.
- Project-first retrieval is an engineering choice: it reduces the chance that a correct but unrelated global fact wins over a project-specific fact.
- This follows the broader pattern of scoped persistent memory used by systems such as Mem0, while ContextBridge keeps the model independent. [P2]

4. What exactly is one memory?

A memory is one small, self-contained piece of durable knowledge. A new agent should be able to understand it without needing the original conversation turn.

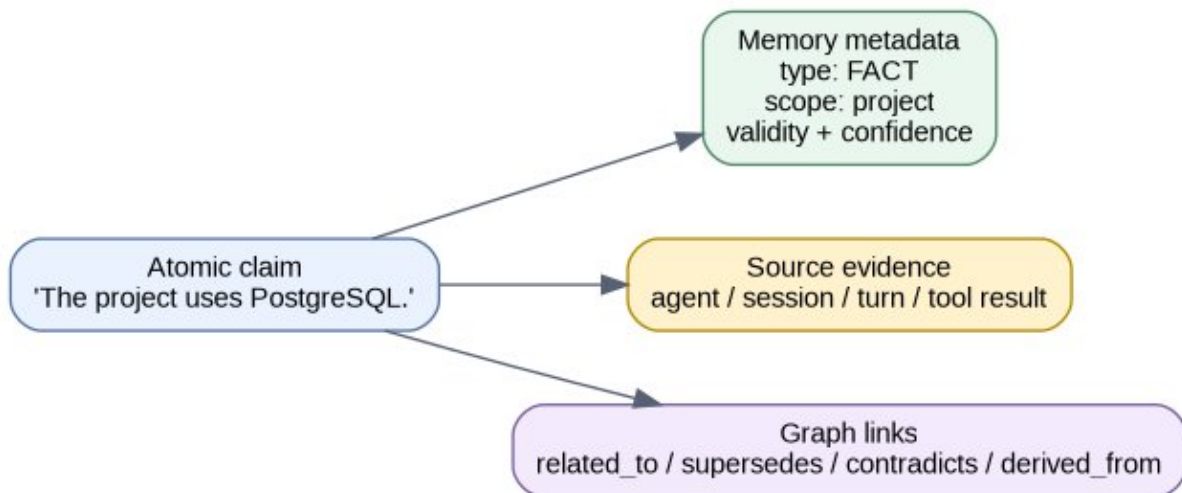


Figure 4. One memory combines a durable claim, metadata, evidence and relationships.

Field	Plain meaning	Reason
id	Unique identity of this memory	Lets us reference, update and link it safely.
type	FACT / DECISION / PREFERENCE / STATE / EVENT / LESSON	Different kinds of information behave differently.
content	The actual claim	This is what retrieval gives to the agent.
scope	Developer or project	Prevents unrelated context from mixing.
valid_from / valid_to	When the claim applies	A fact can be true only during a period.
status	ACTIVE / SUPERSEDED / REJECTED / DELETED	Makes state explicit instead of relying on text.
confidence	How strongly supported it is	Useful when multiple candidates compete.
source_* + evidence_ref	Who/when/where it came from	Makes memory auditable and helps trust decisions.
supersedes_id + graph links	How this memory relates to others	Preserves history and relationships.

Why atomic records? A-MEM uses atomic self-contained notes, and the survey emphasizes that memory formation should turn raw interaction information into useful persistent units. [P1][P16]

5. What we will remember in V1

Type	Meaning	Real example
FACT	Stable information	"The project uses PostgreSQL."
DECISION	A chosen design decision	"Use MCP as the external agent interface."
PREFERENCE	Developer preference	"Prefer TypeScript for frontend code."
STATE	A value that changes	"The authentication migration is pending."
EVENT	Important occurrence	"Agent A completed the schema migration."
LESSON	Reusable learning	"Vector-only search missed exact file names."

Why these six? The survey separates factual and experiential information, while HaluMem evaluates several memory categories and StateMem emphasizes changing state. These six give us a practical V1 without forcing every conversation into a complex schema. [P1][P12][P7]

Procedural workflow memory is not a V1 core feature. AWM and similar work suggest it is useful, but we can add it after the basic factual/state memory pipeline is stable. [P18]

6. Capture first, process later

We do not run an expensive LLM memory pipeline after every single message. We first keep a cheap interaction buffer.

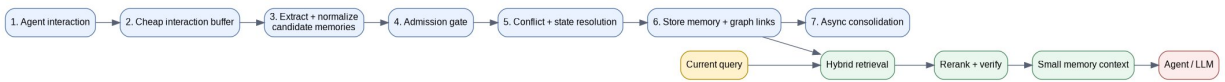
Real-life example: while you are debugging for 20 minutes, most turns are not worth permanently storing. The buffer keeps them available until the system has enough context to decide what matters.

Why? RecMem keeps every interaction in a cheap lower-level memory and triggers richer consolidation selectively. LightMem also separates lightweight short-term handling from more expensive long-term maintenance. [P5][P13]

Trigger	Example	Action
Explicit remember	"Remember that production DB is PostgreSQL."	Process immediately.
Session end	User finishes a coding session.	Process buffered interaction asynchronously.
Idle / batch	No activity for a period.	Process without blocking normal interaction.
No useful signal	"Thanks" / "okay"	Keep only in buffer; usually do not create long-term memory.

Engineering rationale: this keeps user-facing latency low. The exact trigger timing is a tunable implementation parameter, not a claim that one paper proved a universal schedule.

7. Extract and normalize candidate memories



The write path highlighted: capture -> extract -> admit -> resolve -> store.

The extraction step answers one question: "What durable information is present here?"

Step	What happens	Why
Find durable information	Ignore obvious conversational filler.	Reduces noise.
Split into atomic candidates	One useful idea per candidate.	Makes retrieval and updates precise.
Resolve references	Turn "this project" into the actual project.	Makes memory self-contained.
Normalize time	Turn relative phrases into usable temporal metadata.	Needed for evolving state and temporal queries.
Attach scope + provenance	Record project/developer scope and source.	Prevents memory mixing and supports auditing.

SimpleMem, Zep and A-MAC all support the value of normalization before storage, especially for references and time. ProMem shows that one-pass extraction can miss information or retain hallucinated facts, so stronger verification should be available for higher-risk candidates. [P4][P3][P4-AMAC][P14]

8. Admission gate: do not store everything

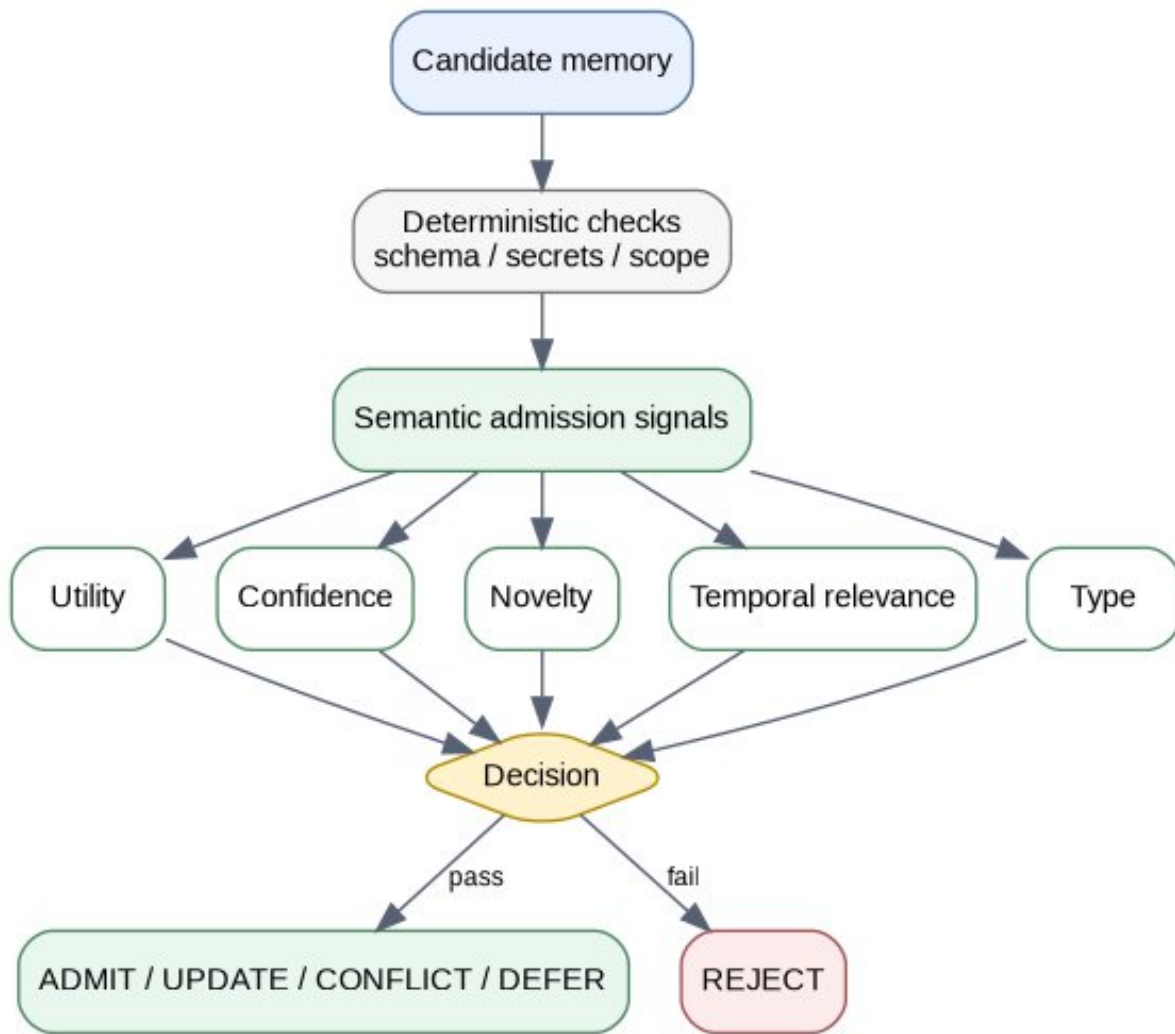


Figure 5. Admission is an explicit decision before long-term storage.

Real-life example: a model may say "You seem to prefer dark mode." That is an inference. It should not become a durable preference just because the model said it.

Our gate therefore considers five signals inspired by A-MAC:

Signal	Plain question	Why we use it
Utility	Could this matter later?	Avoid storing trivial details.
Confidence	Is the source strong enough?	Reduce unsupported or hallucinated memory.
Novelty	Is this already known?	Avoid duplicates.
Temporal relevance	Is the information still relevant in time?	Old/transient information should not dominate.
Type	What kind of memory is it?	Stable preferences and identity facts behave differently from transient state.

Important: we are adopting the idea of a multi-signal gate, not blindly copying A-MAC weights or its threshold. Those parameters must be tuned for ContextBridge and evaluated. A-MAC was designed as an admission-control decision problem and explicitly separates candidate creation from admission. [P4-AMAC]

9. Admission outcomes

Outcome	Meaning	Example
ADMIT	New durable memory	"Project uses Neon Postgres."
UPDATE	New information changes an existing memory	"We moved from MySQL to PostgreSQL."
CONFLICT	Two plausible memories compete	Two agents give different database decisions.
DEFER	Not enough evidence yet	A likely preference is mentioned once but is ambiguous.
REJECT	Not worth storing / unsafe / unsupported	"Thanks" or an unsupported inference.

This separation is also supported by HaluMem and TRUSTMEM: memory extraction, updating, and the trustworthiness of memory transitions are distinct failure points. [P12][P6]

10. Conflict handling: the hard part is not retrieval alone

Suppose the store contains:

Memory	When	Meaning
M1	June	"The project uses PostgreSQL."
M2	September	"The project uses MongoDB."

A vector search may retrieve both. That does not tell us which one is current. The system must understand time, scope, conditions and provenance.

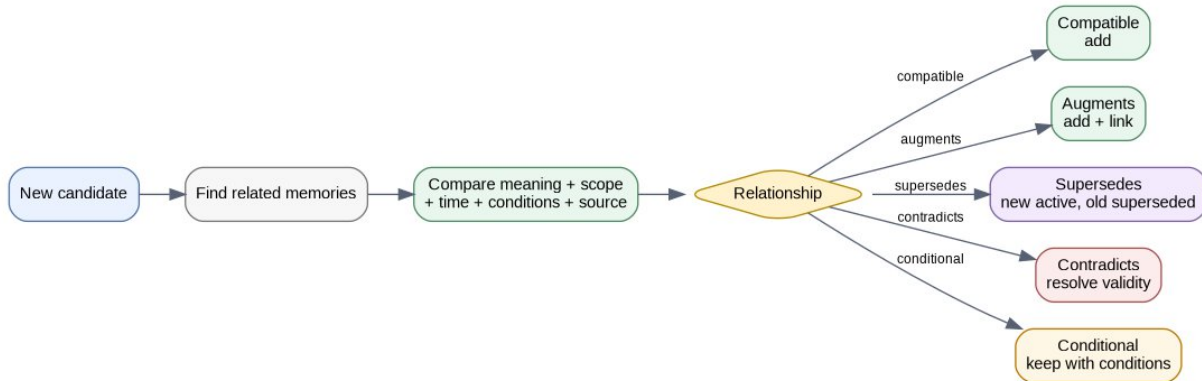


Figure 6. New information is compared with related memories before committing a state.

Why this decision?

- StateMem shows that better retrieval alone does not guarantee correct current state; state tracking needs explicit structure. [P7]
- MemConflict shows that conflicts can be dynamic, static or conditional, and that a valid memory can be retrieved but ranked too low. [P8]
- MOSAIC checks conflicts at save time, reducing the chance that contradictions silently accumulate. [P6-MOSAIC]
- TOKI treats contradiction resolution as a write-time data-management problem and preserves history with temporal structure rather than blind overwrite. [P7-TOKI]

Engineering rule: do not use a simple "latest wins" rule. Newer information is evidence, not automatic truth. We compare applicability, source support, time and relationship.

11. State and history: current truth + historical truth

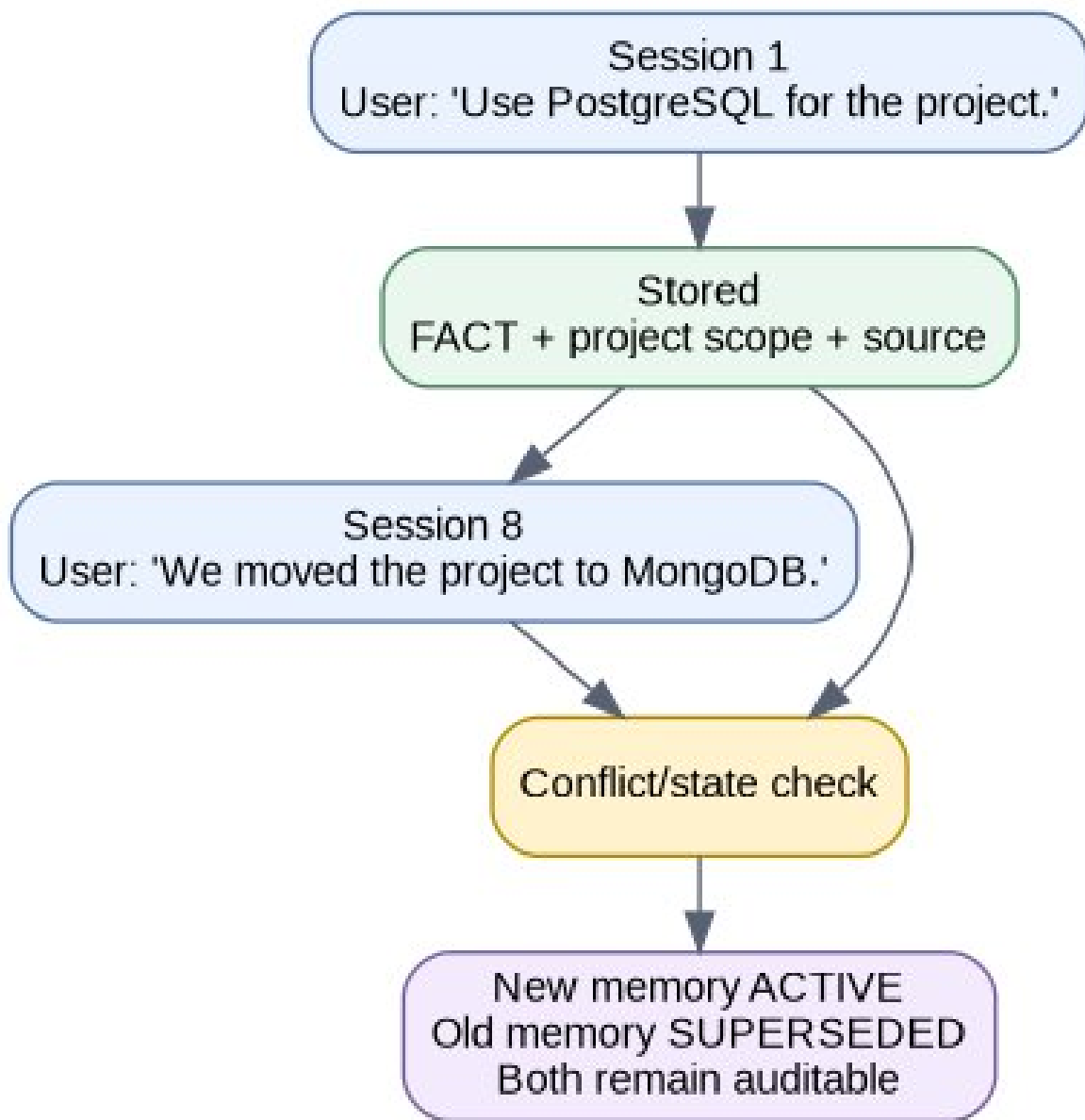


Figure 7. A changed project fact keeps the old version as history instead of deleting it.

The practical V1 rule is:

- The newest applicable memory becomes ACTIVE.
- The replaced memory becomes SUPERSEDED rather than disappearing.
- The relationship is stored with supersedes_id and a graph edge.
- Retrieval normally prefers ACTIVE, currently valid memories.

This gives us two benefits: the agent gets the current state, while the team can still inspect how the state changed. The design is a simplified implementation of ideas from StateMem and TOKI rather than a full copy of either system. [P7][P7-TOKI]

12. Why we chose a lightweight graph in V1

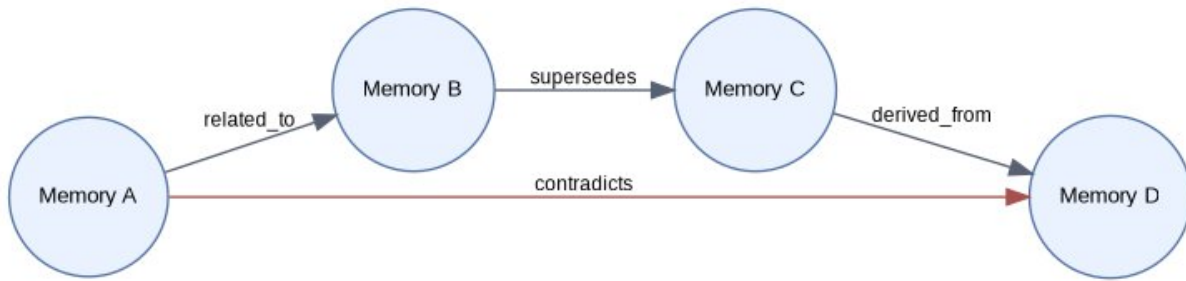


Figure 8. The graph is intentionally small: it stores relationships, not a full GraphRAG system.

We are using a minimal relationship graph with four core edge types:

Edge	Meaning	Example
related_to	Two memories are semantically connected	PostgreSQL decision <-> migration lesson
supersedes	One memory replaces an older state	MongoDB supersedes PostgreSQL
derived_from	Memory was derived from another memory/event	Lesson derived from migration failure
contradicts	Two memories contain incompatible claims	Agent A decision <-> Agent B decision

Why implement it now? Because the relationships already affect other layers: conflict resolution needs them, supersession needs them, provenance benefits from them, and retrieval can optionally expand through them. If we add them only later, the data model would need restructuring.

Why keep it lightweight? Zep, A-MEM and MOSAIC show how graphs can support temporal and relational reasoning, but they also use much richer graph structures. For V1 we want relationship storage without taking on a full knowledge-graph or GraphRAG implementation. [P3][P16][P6-MOSAIC]

Engineering rationale: a graph edge can initially be represented with an ordinary relationship table. We do not need a dedicated graph database in V1.

13. Consolidation: expensive work happens when justified

Consolidation means combining related memories so the store becomes easier to use. It is different from admission and different from conflict resolution.

Example: three sessions repeatedly mention the same authentication decision. Instead of keeping three disconnected notes forever, the system can form a clearer higher-level representation while preserving the source memories.

Why async? LightMem moves expensive maintenance to "sleep time" and RecMem triggers deeper consolidation when repeated information justifies the cost. [P13][P5]

Why not use recurrence alone? A unique but critical fact may happen only once. RecMem itself notes recurrence is only a proxy for salience. Therefore ContextBridge keeps rare facts in the interaction/source layer and allows other signals, especially explicit admission, to create durable memory. [P5]

14. Retrieval: one search method is not enough

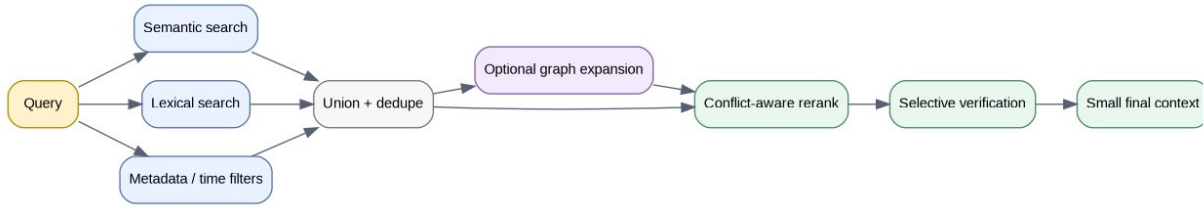


Figure 9. ContextBridge retrieval uses multiple signals and a small final context.

Think about a query such as: "What database are we using in the authentication service, and when did we switch?" A pure keyword search may miss semantically related wording. A pure vector search may miss exact names. A time filter can remove memories that are no longer valid.

Retrieval signal	What it is good at	Failure it helps reduce
Semantic	Meaning similarity	Different wording for the same idea
Lexical	Exact terms, identifiers, file names	Missing exact names or symbols
Metadata / time	Scope, type, dates, validity	Wrong project or stale state
Graph expansion	Related facts and multi-hop links	Missing relationships between memories

SimpleMem explicitly combines semantic, lexical and symbolic retrieval. Zep and MOSAIC add graph-aware retrieval. MemConflict shows that retrieval must also account for conflict validity and ranking, not only similarity. [P4][P3][P6-MOSAIC][P8]

Outside-paper engineering basis: SQLite FTS5 supports full-text search, while pgvector supports vector similarity and can be combined with PostgreSQL full-text search for hybrid retrieval. These make a single-database implementation practical for a V1 prototype. [E1][E2]

15. Adaptive retrieval depth: small first, expand only when needed

We do not want a fixed rule such as "always retrieve 20 memories." More context is not automatically better.

LOCOMO found that a small top-k can perform well in its reported experiments, while Lost-in-the-Middle shows that models can struggle when useful information is buried inside long contexts. SimpleMem also adapts retrieval depth to query complexity. [P11][P19][P4]

ContextBridge V1 therefore starts with a small candidate set and expands when the query is complex, ambiguous or low-confidence. The initial numbers (for example 3 -> 5) are engineering starting points, not a universal optimum.

16. Conflict-aware reranking

After retrieval, we rank candidates again using more than similarity.

Ranking factor	Why it matters
Relevance	Is this about the current query?
Scope	Is this for the right project/developer?
Temporal validity	Is the memory valid for the time being asked about?
Applicability	Does the condition match the current situation?
Status	Prefer ACTIVE over SUPERSEDED unless history is requested.
Provenance + confidence	Prefer better-supported memory when candidates conflict.

MemConflict is the key reason for this layer: the correct memory may be present but lose the ranking competition against a distractor or stale memory. [P8]

17. Verification is selective, not automatic on every query

CRAG and SELF-RAG both show a general pattern: retrieval should be evaluated before it is blindly trusted. [P9][P22]

For ContextBridge, verification is triggered mainly when retrieved memories are ambiguous, conflicting, weakly supported, or important enough to justify extra cost.

This avoids two bad extremes:

- No verification at all -> a stale or conflicting memory can be treated as truth.
- Heavy verification on every query -> unnecessary latency and cost.

The verifier can check whether the selected memory is relevant and whether the proposed answer is actually supported by the retrieved evidence. This is a project-level adaptation of CRAG/SELF-RAG ideas, not a claim that those papers were persistent-memory systems.

18. Context assembly: give the agent a few useful memory cards

The final context should not look like a database dump. Each memory should be a compact card containing the claim plus enough metadata to explain why it is relevant.

Example memory card:

Field	Value
Memory	The authentication service uses Clerk.
Type	FACT
Scope	RozgaarSetu
Validity	Active
Source	Agent B, session 14
Confidence	High
Related	Authentication migration decision

Why compact? Lost-in-the-Middle shows that more context can reduce effective use, and SWE-agent shows that interfaces and context management should be designed around the model rather than exposing unnecessary low-level information. [P19][P14]

19. Provenance: every durable memory should answer "where did this come from?"

ContextBridge keeps `source_agent`, `source_session`, `source_interaction` and `evidence_ref` in the memory model.

Real-life example: two agents propose different authentication libraries. Instead of storing "the project uses X" with no explanation, the record keeps which agent/session/turn produced the claim. Later we can inspect the evidence before deciding whether to keep, supersede or reject it.

Why? The trustworthy-memory survey emphasizes auditability, access control and verifiable updates. MOSAIC stores source/evidence-related metadata, and TOKI emphasizes provenance and reproducibility around contradiction resolution. [P25][P6-MOSAIC][P7-TOKI]

20. Security and memory boundaries

Rule	Reason
Local-first V1	Reduce unnecessary exposure of developer/project memory.
Redact obvious secrets	Long-term memory should not become a secret store.
Keep scope explicit	Prevent project-to-project leakage.
Support inspect/edit/delete in the data model	Users should be able to control persistent memory.
Do not trust model output automatically	Model-generated claims can be unsupported or wrong.

Outside-paper basis: the current MCP specification emphasizes user consent, privacy, access control and careful handling of tool/data exposure. This supports a local-first and user-controlled integration posture. [E3]

21. What the agent actually needs to call

The agent should see a small, stable interface rather than database internals.

Operation	Purpose
<code>memory.search(query, scope, filters)</code>	Find relevant memory.

Operation	Purpose
memory.write(candidate, source)	Explicitly save a memory when requested.
memory.inspect(memory_id)	Inspect why a memory exists and what it relates to.

Automatic lifecycle hooks can handle session capture and background processing, while MCP can expose a standard external interface to agents. The current MCP specification defines tools/resources as standardized ways for applications to share context and capabilities with LLM systems. [E3]

Why hybrid rather than MCP-only? MCP is an interface protocol; it does not by itself guarantee that every conversation turn will be captured automatically. The lifecycle capture path and the agent-facing interface solve different problems.

22. Full real-life walkthrough: a decision changes

Scenario:

5. Agent A is helping build a web app. The team decides to use PostgreSQL.
6. ContextBridge captures the interaction and extracts: FACT "Project uses PostgreSQL."
7. The admission gate accepts it because it is useful, explicit, project-scoped and supported by the source.
8. Later, Agent B records: "We moved this project to MongoDB."
9. ContextBridge finds the earlier PostgreSQL memory, compares meaning, scope and time, and identifies a state change.
10. The MongoDB memory becomes ACTIVE. PostgreSQL becomes SUPERSEDED, not deleted.
11. The graph records the supersedes relationship.
12. Later, a new agent asks "What database are we using?" Hybrid retrieval finds both candidates, but status/time-aware reranking prefers MongoDB.
13. Only the relevant current memory card is given to the agent.

This single example explains why storage, graph links, temporal metadata, conflict resolution and retrieval cannot be designed as isolated components.

23. Another real-life walkthrough: two agents disagree

Scenario: Agent A recommends Clerk. Agent B recommends Auth0. Neither recommendation is automatically "truth".

14. Both candidate decisions are stored with provenance instead of pretending the model output is authoritative.
15. The conflict layer marks them as competing candidates.
16. A condition or explicit human decision can later resolve the conflict.
17. If the team chooses Clerk, the chosen decision becomes the active decision and the competing candidate is retained as history/alternative evidence.

This is safer than silently keeping whichever agent happened to write last.

24. Why not store raw code as memory?

The purpose of ContextBridge is durable project knowledge, not another source-code database. Full source code changes frequently and is already available from the repository/workspace.

Therefore, memory should store things such as:

- "The authentication middleware is in src/auth/middleware.ts."
- "The team decided to use Clerk because it simplifies session management."
- "The migration is pending."

For code-related memories we keep file/symbol/commit references where useful. This follows the broader distinction between durable knowledge and the live working environment.

25. Why we are not copying the most complex graph-memory papers

Zep, A-MEM, ProMem and MOSAIC show useful graph-based organization, but their complete architectures are much richer than what we need for a V1 prototype. [P3][P16][P14][P6-MOSAIC]

Our decision is deliberately narrower: store a small relationship graph now so the data model is future-proof, but delay advanced graph algorithms, community detection, full temporal knowledge-graph reasoning and other complexity until evaluation shows they are needed.

26. Design decisions - one-page traceability map

Decision	Reason in simple language	Main basis
Developer + project scopes	Share durable context without mixing unrelated projects.	P2 + engineering
Atomic memory records	One memory should stand alone and be easy to retrieve/update.	P1, P16
Six V1 memory types	Enough structure for facts, choices, preferences, state, events and lessons without over-modeling.	P1, P7, P12
Cheap interaction buffer	Do not pay LLM processing cost after every message.	P5, P13
Async automatic memory creation	Keep the user-facing path fast.	P5, P13 + engineering
Immediate explicit remember path	Respect a direct user request instead of waiting for a batch.	Engineering + user-control principle
Normalization of time/references	Make memories self-contained and time-aware.	P4, P3, P4-AMAC
Admission gate	Do not store everything the model says.	P4-AMAC, P12
Five admission signals	Balance usefulness, support, duplication, time and durability.	P4-AMAC
Conflict detection at write time	Stop contradictions from silently accumulating.	P6-MOSAIC, P7-TOKI
Explicit state + supersession	Current truth should be separate from historical truth.	P7, P8, P7-TOKI
Lightweight graph in V1	Relationships already affect conflict, provenance and retrieval; adding later would require schema changes.	P3, P16, P6-MOSAIC + engineering
No graph database requirement	Need graph relationships, not full graph infrastructure.	Engineering
Async consolidation	Use expensive organization only when useful.	P5, P13
Hybrid retrieval	Different search signals solve different failure modes.	P4, P3, P6-MOSAIC
Small/adaptive retrieval set	More context can add noise; complexity varies by query.	P11, P19, P4
Conflict-aware reranking	Correct memory can be present but lose to a stale/distractor memory.	P8
Selective retrieval verification	Check retrieved evidence when uncertainty is high without adding cost to every query.	P9, P22
Compact memory cards	Long prompts can hurt information use.	P19, P14
Provenance in every record	We need to know who/when/where produced the memory.	P25, P6-MOSAIC, P7-TOKI
Local-first V1	Reduce exposure and simplify deployment.	Engineering + E3
Simple MCP-facing API	Agents need a small stable interface, not database internals.	P14 + E3

27. What is deliberately NOT in the V1 methodology?

Not in V1	Why
Full procedural memory / workflow learning	Useful, but not needed before factual/state memory is reliable.
Full community-based graph reasoning	Adds significant complexity before we know it is needed.
A dedicated graph database	Relationship tables are sufficient to start.
LLM verification on every write/query	Too expensive; use targeted checks.
Automatic deletion based only on age/access count	Rare information can still be important.
Blind latest-wins updates	Can preserve the wrong state.
Raw conversation as the only long-term representation	Too noisy and difficult to maintain.
Full source code in long-term memory	The repository/workspace already owns source code.

28. How we will evaluate whether the methodology actually works

The methodology is not finished when the pipeline runs. We need to test the pipeline itself.

Layer	Example question
Extraction	Did we extract the important memory?
False-memory resistance	Did we avoid storing information that was not supported?
Updating	When a value changed, did we update it correctly?
Conflict resolution	Did the correct memory remain active?
Retrieval	Was the supporting memory retrieved?
Ranking	Did the valid memory beat a stale/distractor memory?
Utilization	Was the retrieved memory actually used correctly?
Task utility	Did the agent complete the real task better?
Efficiency	How much latency/token cost did the memory system add?

This comes directly from the evaluation direction in HaluMem, MemConflict, MemoryAgentBench and MemoryArena: end-to-end answer quality alone cannot tell us which memory layer failed. [P12][P8][P20][P21]

29. The three questions every team member should remember

When we discuss a new feature, ask these three questions:

18. What durable knowledge should exist after this interaction?
19. What should happen if that knowledge later changes or conflicts?
20. What is the smallest amount of that knowledge the agent needs for its current task?

These map to ContextBridge's three central concerns: formation, evolution and retrieval.

30. Reference key

Tag	Source used in this guide
P1	Memory in the Age of AI Agents: A Survey
P2	Mem0: Building Production-Ready AI Agents with Scalable Long-Term Memory
P3	Zep: A Temporal Knowledge Graph Architecture for Agent Memory
P4	SimpleMem: Efficient Lifelong Memory for LLM Agents
P4-AMAC	Adaptive Memory Admission Control for LLM Agents (A-MAC)
P5	RecMem: Recurrence-based Memory Consolidation for Efficient and Effective Long-Running LLM Agents
P6	TRUSTMEM: Learning Trustworthy Memory Consolidation for LLM Agents with Long-Term Memory
P6-MOSAIC	Accurate and Efficient Long-Term Memory for LLM Agents (MOSAIC)
P7	Can Agent Memory Systems Track Evolving State? (StateMem)
P7-TOKI	TOKI: A Bitemporal Operator Algebra for Contradiction Resolution in LLM-Agent Persistent Memory
P8	MemConflict: Evaluating Long-Term Memory Systems Under Memory Conflicts
P9	Corrective Retrieval Augmented Generation (CRAG)
P11	Evaluating Very Long-Term Conversational Memory of LLM Agents (LOCOMO)
P12	HaluMem: Evaluating Hallucinations in Memory Systems of Agents
P13	Reproducing LightMem: Naive RAG Is Just as Good for Memory Management
P14	Beyond Static Summarization: Proactive Memory Extraction for LLM Agents (ProMem)
P16	A-MEM: Agentic Memory for LLM Agents
P18	AWM: procedural-memory-related work
P19	Lost in the Middle: How Language Models Use Long Contexts
P20	MemoryAgentBench: Evaluating Memory in LLM Agents via Incremental Multi-Turn Interactions
P21	MemoryArena: Benchmarking Agent Memory in Interdependent Multi-Session Agentic Tasks
P22	SELF-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection
P25	Memory in the Age of AI Agents: Trustworthy-memory/security discussion
E1	SQLite FTS5 official documentation - full-text search
E2	pgvector official documentation - vector similarity + hybrid search patterns
E3	Model Context Protocol Specification - standardized context/tools interface and security principles

Outside-paper references

[E1] **SQLite FTS5**: Official documentation: <https://www.sqlite.org/fts5.html>

[E2] **pgvector**: Official documentation / repository: <https://github.com/pgvector/pgvector>

[E3] **Model Context Protocol**: Current specification: <https://modelcontextprotocol.io/specification/latest>

31. Final team mental model

ContextBridge is not just a vector database. It is a memory lifecycle:

CAPTURE -> EXTRACT -> ADMIT -> RESOLVE -> STORE -> CONSOLIDATE -> RETRIEVE -> VERIFY -> PRESENT

Each stage exists because a different failure can happen. The architecture stays intentionally simple where possible, but it preserves the data needed for future graph reasoning, stronger verification and better conflict handling.

Methodology baseline: v1.0. Implementation details such as exact database schema, embedding model, thresholds, weights, queue technology and deployment topology remain tunable and should be validated during the next design phase.